Informe técnico

Implementación del modelo físico de la Red Social Estudiantil Pascualina

Tarea 3 - Lenguaje de definición de datos (DDL) en PostgreSQL

Autor

Harold Stevent Suarez Palacio

Bases de Datos I

Docente: Oralia Cortés Grajales

Instituto Universitario Pascual Bravo

Octubre de 2026

Contenido

1. Introducción y alcance

2. Selección del SGBD y justificación

3. Del modelo lógico al modelo físico

4. Convenciones de nomenclatura y estándares aplicados

5. Descripción de las tablas implementadas

6. Implementación de las relaciones entre tablas

7. Restricciones de integridad e índices

8. Estructura del script DDL y verificación

9. Organización del repositorio Git

10. Conclusiones individuales

11. Referencias

1. Introducción y alcance

Las plataformas institucionales de la universidad son formales y no favorecen la conexión casual entre estudiantes, el intercambio de conocimiento entre pares, la formación de grupos de estudio ni la organización de eventos informales. La Red Social Estudiantil Pascualina busca cubrir esa necesidad, y este informe documenta la **implementación física** de la base de datos que la soporta.

La base de datos debe permitir cinco capacidades: (1) **crear perfiles** con área de estudio, intereses y habilidades; (2) **conectar con compañeros**, incluyendo mentoría de cursos avanzados; (3) **publicar actualizaciones** (preguntas, recursos, noticias, memes); (4) **crear y unirse a grupos** de estudio, hackatones y clubes; y (5) **programar eventos**. El modelo físico resultante tiene **12 tablas, 18 claves foráneas, 12 claves primarias (7 simples y 5 compuestas) y 14 índices adicionales**, y está contenido en el script `Tarea3/red_social_pascualina_ddl.sql`.

2. Selección del SGBD y justificación

Se eligió **PostgreSQL 15 o superior** (la verificación se hizo sobre la versión 16) administrado con pgAdmin 4. PostgreSQL es un SGBD objeto-relacional de código abierto, con licencia permisiva, que cumple en gran medida el estándar SQL y tiene más de 30 años de desarrollo. Para este proyecto resultan relevantes las siguientes características:

- **Integridad declarativa completa:** claves primarias y foráneas con ON DELETE/ON UPDATE, restricciones CHECK, UNIQUE y NOT NULL, que se aplican en el motor y no dependen de la aplicación.
- **Columnas de identidad estándar SQL** (`GENERATED ALWAYS AS IDENTITY`) y tipos enteros de 2, 4 y 8 bytes (SMALLINT, INTEGER, BIGINT) para dimensionar cada clave según el volumen esperado.
- **Tipos temporales con zona horaria** (`TIMESTAMPTZ`), útiles porque los eventos se programan y se consultan desde dispositivos distintos.
- **Esquemas (schemas)** independientes de la base de datos, lo que permite agrupar los objetos de la red social en el esquema `red_social`.
- **Escalabilidad funcional:** índices parciales, compuestos y de texto completo (GIN), JSONB y consultas recursivas, que serán útiles si la red crece (búsqueda de publicaciones, recomendación de compañeros).
- **Costo cero y portabilidad:** corre en Windows, Linux y macOS, sin licencias, y existe en todos los proveedores de nube.

Alternativas consideradas

Criterio	PostgreSQL (elegido)	MySQL 8 / MariaDB	Microsoft SQL Server
Licencia y costo	Open source (licencia PostgreSQL). Sin costo.	MySQL: GPL con edición comercial de Oracle; MariaDB: GPL. Sin costo en edición comunitaria.	Propietario. La edición Express es gratuita pero limitada (por ejemplo, 10 GB por base de datos); las demás requieren licencia.
Cumplimiento del estándar SQL	Muy alto: identidad estándar, CHECK, esquemas, CTE recursivas, ventanas.	Bueno. CHECK solo se aplica desde MySQL 8.0.16 (antes se aceptaba y se ignoraba). Sin índices parciales.	Alto, con dialecto propio (T-SQL).
Integridad referencial	Motor único con FK transaccionales, acciones RESTRICT, CASCADE, SET NULL, SET DEFAULT, NO ACTION; FK diferibles (DEFERRABLE).	InnoDB soporta FK; MyISAM no. Cuidado con el motor elegido. No hay FK diferibles.	Soporta FK y acciones; limita rutas de cascada múltiples (error 'multiple cascade paths').
Tipos de datos	TIMESTAMPTZ, BOOLEAN real, TEXT sin penalización, JSONB, arreglos, ENUM, rangos.	TIMESTAMP limitado a 2038 (DATETIME no guarda zona); BOOLEAN es alias de TINYINT(1).	DATETIMEOFFSET, BIT (no BOOLEAN), NVARCHAR/VARCHAR(MAX).
Esquemas	Un esquema es un espacio de nombres dentro de la base de datos.	'Esquema' es sinónimo de base de datos; no hay espacio de nombres adicional.	Esquemas soportados (dbo, etc.).
Herramienta gráfica	pgAdmin 4 (gratuita, ofrecida en el curso).	MySQL Workbench (gratuita).	SSMS / Azure Data Studio.
Ventajas para este proyecto	Integridad fuerte, tipos adecuados, gratuito, material del curso orientado a PostgreSQL.	Muy difundido, fácil de instalar, buen rendimiento en lecturas simples.	Excelente tooling y soporte empresarial.
Desventajas para este proyecto	Configuración inicial algo más exigente; consumo de memoria mayor que MySQL en instancias pequeñas.	Menos garantías de integridad si se usa mal el motor; funcionalidades avanzadas más limitadas.	Licenciamiento, mayor peso, y no es el SGBD de referencia del curso.

Otras opciones descartadas: **SQLite**, por ser un archivo embebido con un único escritor concurrente y claves foráneas desactivadas por defecto (no apropiado para una red social con muchos usuarios simultáneos); y **Oracle Database**, por costo de licencia y complejidad de administración desproporcionada para el alcance. En conclusión, PostgreSQL ofrece la mejor combinación de integridad referencial estricta, tipos de datos adecuados al dominio (fechas con zona horaria, booleanos) y costo cero, y además coincide con el material de consulta de la unidad (Canepa, 2016; Hsu y Obe, 2015).

3. Del modelo lógico al modelo físico

El punto de partida es el modelo lógico normalizado de la **Tarea 2**, que definió cuatro tablas (**Usuario, Publicacion, Grupo y Evento**) con su diccionario de datos. Ese modelo ya anticipaba, en su análisis y en la normalización, tres necesidades que requieren tablas adicionales: (1) en la 1FN se separaron los campos multivalor *intereses_usuario* y *miembros_grupo* 'en registros individuales'; (2) en la 3FN la carrera se independizó del perfil del estudiante para no repetirla; y (3) el análisis de necesidades incluía seguir a otros estudiantes y establecer mentoría. El modelo físico conserva las cuatro tablas originales con los mismos nombres de campo y agrega las tablas que esas decisiones implican. Resumen de la trazabilidad:

Modelo lógico (Tarea 2)	Modelo físico (Tarea 3)	Observación
Usuario(id_usuario, nombre, correo, contrasena, area_estudio)	usuario(id_usuario, nombre, correo, contrasena, id_programa_academico, + semestre, biografia, foto_url, activo, fecha_registro)	area_estudio (texto) pasa a FK hacia programa_academico (3FN). Se agrega semestre para identificar mentores de cursos avanzados.

Modelo lógico (Tarea 2)	Modelo físico (Tarea 3)	Observación
Publicacion(id_publicacion, contenido, fecha_creacion, id_usuario)	publicacion(id_publicacion, id_usuario, id_grupo, tipo, titulo, contenido, url_recurso, fecha_creacion)	Mismos campos del modelo lógico; tipo y url_recurso distinguen pregunta, recurso, noticia o meme; id_grupo opcional.
Grupo(id_grupo, nombre_grupo, descripcion)	grupo(id_grupo, id_creador, nombre_grupo, descripcion, tipo, es_privado, fecha_creacion)	Mismos campos; se agrega el creador y el tipo (estudio, hackatón, club).
Evento(id_evento, titulo_evento, fecha_hora, id_grupo)	evento(id_evento, id_organizador, id_grupo, titulo_evento, descripcion, tipo, lugar, enlace_virtual, fecha_hora, fecha_fin, cupo_maximo)	Mismos campos; id_grupo es opcional porque también hay actividades sociales sin grupo; se agrega organizador (coordinador).
1FN: se separa 'intereses_usuario'	interes + usuario_interes	Catálogo y tabla asociativa N:M.
1FN: se separa 'miembros_grupo'	miembro_grupo	Asociativa N:M usuario-grupo con rol.
3FN: carrera independizada	programa_academico	Catálogo referenciado desde usuario.
Análisis: seguir y mentoría	seguimiento	Relación reflexiva usuario-usuario con indicador de mentoría.
Interacción en publicaciones (análisis de necesidades)	comentario, reaccion, asistencia_evento	Soportan responder, valorar publicaciones y confirmar asistencia a eventos.

Para pasar al modelo físico se aplicaron además estas reglas:

Elemento del modelo lógico	Transformación física	Resultado
Entidad fuerte	Una tabla por entidad, con clave primaria sustituta (identidad).	usuario, grupo, publicacion, comentario, evento
Atributo multivalorado (intereses, habilidades, deportes)	Se extrae a un catálogo y a una tabla asociativa, para cumplir la primera forma normal.	interes + usuario_interes
Dominio repetido (área de estudio)	Se convierte en tabla catálogo referenciada por FK, evitando redundancia y errores de digitación.	programa_academico
Relación N:M	Tabla asociativa con clave primaria compuesta por las dos claves foráneas; los atributos propios de la relación pasan a la tabla.	usuario_interes, miembro_grupo, asistencia_evento, reaccion
Relación reflexiva N:M (seguir / mentoría)	Tabla asociativa con dos FK a usuario (seguidor y seguido) y un CHECK que prohíbe seguirse a sí mismo.	seguimiento
Relación 1:N	La clave foránea se coloca en el lado 'muchos'.	usuario -> publicacion, publicacion -> comentario, grupo -> evento, etc.
Relación opcional (0..1)	Clave foránea que admite NULL.	publicacion.id_grupo, evento.id_grupo
Dominio de valores cerrado	Restricción CHECK con la lista de valores permitidos.	tipo, rol, estado, nivel_habilidad

Resultado: el esquema está en **tercera forma normal (3FN)**: todos los atributos dependen de la clave completa y solo de ella; los datos repetidos (programas, intereses) viven en catálogos. Los atributos derivados (número de seguidores, número de reacciones, cupo disponible) **no se almacenan**; se calculan por consulta para evitar inconsistencias.

Nota: las tablas son 12 en total. Las 5 tablas de 'entidad' son usuario, grupo, publicacion, comentario y evento; las 2 de catálogo son programa_academico e interes; y las 5 asociativas son usuario_interes, seguimiento, miembro_grupo, reaccion y asistencia_evento.

4. Convenciones de nomenclatura y estándares aplicados

Regla	Aplicación en el script
Minúsculas y snake_case, sin tildes, sin espacios ni palabras reservadas	Todos los identificadores (p. ej. nombre_grupo, fecha_creacion). Se evitan 'user' y 'group' por ser palabras reservadas; por eso se usan usuario y grupo en español, y 'contrasena' se escribe sin eñe.
Nombres de tabla en singular	usuario, grupo, publicacion, evento (cada fila representa una instancia).
Clave primaria: id_	id_usuario, id_grupo, id_publicacion, id_evento, id_comentario, id_interes, id_programa_academico.
Clave foránea con el mismo nombre que la clave que referencia	usuario.id_programa_academico referencia programa_academico.id_programa_academico. Excepción justificada: cuando una tabla referencia dos veces a usuario o el rol importa, se usa un nombre con rol (id_creador, id_organizador, id_seguidor, id_seguido).
Restricciones con nombre explícito y prefijo	pk_, fk__, uq__, ck__. Facilitan leer los mensajes de error y modificar la restricción con ALTER TABLE.
Índices con prefijo ix_	ix_publicacion_usuario, ix_evento_fecha_hora, etc.
Tipos y tamaños explícitos	Cada VARCHAR lleva longitud; cada entero se dimensiona (SMALLINT, INTEGER, BIGINT) según el volumen esperado.
Esquema propio	Todos los objetos viven en el esquema red_social, no en public.
Orden de creación	Primero tablas sin dependencias (catálogos), luego las que dependen de ellas, y al final los índices; así el script se ejecuta sin errores de referencia.

5. Descripción de las tablas implementadas

Para cada tabla se indica el propósito, los campos con su tipo y tamaño, y la justificación de los tipos de datos elegidos. PK = clave primaria, FK = clave foránea, UQ = única, NN = NOT NULL.

Tabla `programa_academico`

Catálogo de carreras o áreas de estudio. Evita escribir el programa como texto libre en cada perfil.

Campo	Tipo de dato	Restricciones	Justificación
id_programa_academico	SMALLINT identidad	PK	Un catálogo de programas no superará 32 767 filas; SMALLINT usa 2 bytes frente a 4 de INTEGER y reduce el tamaño de la FK en usuario.
nombre	VARCHAR(100)	NN, UQ	Los nombres de programa son cortos y de longitud acotada; UQ evita duplicados.
facultad	VARCHAR(100)	NULL	Dato descriptivo opcional.

Clave primaria: id_programa_academico. Clave sustituta numérica: el nombre del programa podría cambiar y es más pesado como clave. El nombre se protege con UNIQUE.

Tabla `usuario`

Perfil del estudiante (requisito 'crear perfiles'); corresponde a la tabla Usuario del modelo lógico. Es la entidad central: casi todas las demás tablas la referencian.

Campo	Tipo de dato	Restricciones	Justificación
id_usuario	BIGINT identidad	PK	Es la tabla más referenciada y la que más crece; BIGINT evita agotar el rango de INTEGER (2 147 483 647) y no obliga a migrar todas las FK más adelante.
id_programa_academico	SMALLINT	NULL, FK	Reemplaza a area_estudio del modelo lógico (opcional, igual que allí). Mismo tipo que la PK del catálogo (las FK deben coincidir en tipo).
nombre	VARCHAR(100)	NN	Nombre completo, tal como en el diccionario de datos de la Tarea 2. VARCHAR con límite en lugar de TEXT porque el límite documenta la regla de negocio.
correo	VARCHAR(100)	NN, UQ	Correo institucional, único por persona; sirve como clave candidata para autenticación.
contrasena	VARCHAR(255)	NN	Se guarda el hash (bcrypt/argon2), nunca la contraseña en texto plano; 255 deja margen a algoritmos con salida larga.
semestre	SMALLINT	NN, CHECK 1-12	Permite identificar mentores de cursos avanzados; el CHECK impide semestres absurdos.
biografia	VARCHAR(500)	NULL	Texto corto de presentación, con límite de longitud.
foto_url	VARCHAR(255)	NULL	Se almacena la ruta/URL; la imagen no se guarda en la base de datos.
activo	BOOLEAN	NN, DEFAULT TRUE	Permite desactivar cuentas sin borrar historial (borrado lógico).
fecha_registro	TIMESTAMPTZ	NN, DEFAULT now	Fecha con zona horaria, asignada por el motor.

Clave primaria: id_usuario. Clave sustituta: el correo podría cambiar, y una clave numérica de 8 bytes es más eficiente en las muchas FK que apuntan a esta tabla. El correo queda como clave candidata (UNIQUE).

Tabla `interes`

Catálogo de intereses, habilidades y deportes (disciplinas, programación competitiva, videojuegos, fútbol...).

Campo	Tipo de dato	Restricciones	Justificación
id_interes	SMALLINT identidad	PK	Catálogo pequeño y estable.
nombre	VARCHAR(60)	NN, UQ	Nombre corto y único del interés.
categoria	VARCHAR(20)	NN, CHECK	Valores: academico, tecnologia, deporte, cultura, videojuegos, otro. Se usa CHECK en lugar de un ENUM de PostgreSQL porque agregar o quitar valores en un CHECK es un simple ALTER TABLE, y el CHECK es portable a otros SGBD.

Clave primaria: id_interes. Clave sustituta numérica; el nombre se protege con UNIQUE.

Tabla `usuario_interes`

Asociativa N:M entre usuario e interés (cada estudiante tiene varios intereses y cada interés lo comparten varios estudiantes).

Campo	Tipo de dato	Restricciones	Justificación
id_usuario	BIGINT	PK, FK	Mismo tipo que usuario.id_usuario.
id_interes	SMALLINT	PK, FK	Mismo tipo que interes.id_interes.
nivel_habilidad	VARCHAR(15)	NN, CHECK, DEFAULT 'basico'	Atributo propio de la relación: basico, intermedio, avanzado.

Clave primaria: (id_usuario, id_interes). Clave compuesta natural: impide repetir el mismo interés en un perfil y no necesita una columna id adicional.

Tabla `seguimiento`

Relación reflexiva N:M de usuario con usuario ('conectar con compañeros'): quién sigue a quién, con la marca de si es una relación de mentoría.

Campo	Tipo de dato	Restricciones	Justificación
id_seguidor	BIGINT	PK, FK	Usuario que sigue.
id_seguido	BIGINT	PK, FK	Usuario seguido.
es_mentoria	BOOLEAN	NN, DEFAULT FALSE	Indica que el seguido actúa como mentor (curso avanzado).
fecha_registro	TIMESTAMPTZ	NN, DEFAULT now	Cuándo se creó el vínculo.

Clave primaria: (id_seguidor, id_seguido). Garantiza que un usuario no sigue dos veces al mismo. CHECK `id_seguidor <> id_seguido` prohíbe el autoseguimiento. La relación es dirigida: seguir no implica ser seguido.

Tabla `grupo`

Grupos de estudio, equipos de hackatón y clubes de interés (tabla Grupo del modelo lógico).

Campo	Tipo de dato	Restricciones	Justificación
id_grupo	INTEGER identidad	PK	Se esperan muchos menos grupos que usuarios: INTEGER es suficiente.
id_creador	BIGINT	NN, FK	Usuario que fundó el grupo.
nombre_grupo	VARCHAR(100)	NN, UQ	Nombre de la materia o interés (igual que en el modelo lógico); único.
descripcion	VARCHAR(255)	NULL	Propósito del grupo, con el mismo tamaño del diccionario de la Tarea 2.
tipo	VARCHAR(20)	NN, CHECK	estudio, hackaton, club, otro.
es_privado	BOOLEAN	NN, DEFAULT FALSE	Grupo abierto o por invitación.
fecha_creacion	TIMESTAMPTZ	NN, DEFAULT now	Auditoría de creación.

Clave primaria: id_grupo. Clave sustituta numérica; nombre_grupo queda como clave candidata (UNIQUE).

Tabla `miembro_grupo`

Asociativa N:M entre usuario y grupo ('unirse a grupos'), con el rol dentro del grupo.

Campo	Tipo de dato	Restricciones	Justificación
id_grupo	INTEGER	PK, FK	Mismo tipo que grupo.id_grupo.
id_usuario	BIGINT	PK, FK	Mismo tipo que usuario.id_usuario.
rol	VARCHAR(15)	NN, CHECK, DEFAULT 'miembro'	admin, moderador, miembro.
fecha_union	TIMESTAMPTZ	NN, DEFAULT now	Cuándo ingresó.

Clave primaria: (id_grupo, id_usuario). Un usuario solo puede pertenecer una vez a cada grupo.

Tabla `publicacion`

Actualizaciones: preguntas sobre tareas, recursos útiles, noticias tecnológicas y memes. Puede pertenecer a un grupo o ser pública.

Campo	Tipo de dato	Restricciones	Justificación
id_publicacion	BIGINT identidad	PK	Es la tabla de mayor crecimiento junto con comentarios y reacciones; BIGINT evita el desbordamiento.
id_usuario	BIGINT	NN, FK	Autor de la publicación; mismo nombre que la PK referenciada, como en el modelo lógico.
id_grupo	INTEGER	NULL, FK	Opcional: NULL indica publicación general.
tipo	VARCHAR(15)	NN, CHECK, DEFAULT 'general'	general, pregunta, recurso, noticia, meme.
titulo	VARCHAR(150)	NULL	Título opcional, acotado.
contenido	TEXT	NN	Se usa TEXT (y no VARCHAR) porque la longitud de un post es impredecible; en PostgreSQL TEXT y VARCHAR tienen el mismo rendimiento, por lo que se reserva VARCHAR para cuando el límite es una regla de negocio.
url_recurso	VARCHAR(255)	NULL	Enlace a un recurso o imagen.
fecha_creacion	TIMESTAMPTZ	NN, DEFAULT now	Permite ordenar la línea de tiempo (mismo nombre que el modelo lógico).

Clave primaria: id_publicacion. Clave sustituta; no existe una combinación natural estable que identifique un post.

Tabla `comentario`

Comentarios de los usuarios sobre una publicación.

Campo	Tipo de dato	Restricciones	Justificación
id_comentario	BIGINT identidad	PK	Alto volumen esperado.
id_publicacion	BIGINT	NN, FK	Publicación comentada.
id_usuario	BIGINT	NN, FK	Usuario que comenta.
contenido	VARCHAR(1000)	NN	Comentarios cortos; el límite protege el diseño de la interfaz.
fecha_creacion	TIMESTAMPTZ	NN, DEFAULT now	Orden cronológico.

Clave primaria: id_comentario. Un usuario puede comentar varias veces la misma publicación, por lo que no sirve una clave compuesta natural; se usa una clave sustituta.

Tabla `reaccion`

Reacciones a una publicación (me gusta, útil, divertido, celebrar).

Campo	Tipo de dato	Restricciones	Justificación
id_publicacion	BIGINT	PK, FK	Publicación reaccionada.
id_usuario	BIGINT	PK, FK	Quien reacciona.
tipo	VARCHAR(15)	NN, CHECK, DEFAULT 'me_gusta'	me_gusta, util, divertido, celebrar.
fecha_registro	TIMESTAMPTZ	NN, DEFAULT now	Registro temporal.

Clave primaria: (id_publicacion, id_usuario). La clave compuesta impone una sola reacción por usuario y publicación; para cambiarla se actualiza el tipo.

Tabla `evento`

Reuniones de estudio, talleres y actividades sociales ('programar eventos'); corresponde a la tabla Evento del modelo lógico.

Campo	Tipo de dato	Restricciones	Justificación
id_evento	INTEGER identidad	PK	Volumen moderado.
id_organizador	BIGINT	NN, FK	Usuario que organiza.
id_grupo	INTEGER	NULL, FK	Grupo que organiza; en el modelo lógico era obligatorio, aquí es opcional para permitir actividades sociales abiertas.
titulo_evento	VARCHAR(150)	NN	Nombre del taller o actividad; mismo tamaño que en el diccionario de la Tarea 2.
descripcion	VARCHAR(1000)	NULL	Detalle.
tipo	VARCHAR(20)	NN, CHECK	estudio, taller, social, otro.
lugar, enlace_virtual	VARCHAR(150), VARCHAR(255)	NULL	Presencial o virtual; CHECK exige al menos uno.
fecha_hora, fecha_fin	TIMESTAMPTZ	fecha_hora NN; fecha_fin NULL; CHECK fin > inicio	fecha_hora es el campo del modelo lógico (inicio del evento); fecha_fin es opcional. Con zona horaria porque los participantes pueden estar en otros husos; el CHECK evita rangos inválidos.
cupo_maximo	SMALLINT	NULL, CHECK > 0	NULL significa sin límite.

Clave primaria: id_evento. Clave sustituta numérica.

Tabla `asistencia_evento`

Asociativa N:M entre usuario y evento: confirmaciones de asistencia.

Campo	Tipo de dato	Restricciones	Justificación
id_evento	INTEGER	PK, FK	Mismo tipo que evento.id_evento.
id_usuario	BIGINT	PK, FK	Mismo tipo que usuario.id_usuario.
estado	VARCHAR(15)	NN, CHECK, DEFAULT 'confirmado'	confirmado, tal_vez, cancelado.
fecha_respuesta	TIMESTAMPTZ	NN, DEFAULT now	Cuándo respondió.

Clave primaria: (id_evento, id_usuario). Un usuario tiene una sola respuesta por evento.

Por qué INTEGER, BIGINT o SMALLINT: el tipo se dimensiona según el crecimiento esperado de cada tabla. SMALLINT (2 bytes, hasta 32 767) para catálogos pequeños; INTEGER (4 bytes, hasta 2 147 483 647) para grupos y eventos; BIGINT (8 bytes) para usuario, publicación y comentario, que son las tablas que más crecen y las más referenciadas. Como una FK debe tener el mismo tipo que la PK que referencia, la elección de la PK se propaga a toda la red de relaciones y es difícil de cambiar después; por eso se decide desde el diseño.

6. Implementación de las relaciones entre tablas

Las 18 claves foráneas se definen dentro del `CREATE TABLE` con nombre explícito. Criterios generales para las acciones referenciales: **ON UPDATE CASCADE** en todas, porque las claves son sustitutas e inmutables y la acción es inocua; **ON DELETE CASCADE** cuando la fila hija no tiene sentido sin el padre (datos dependientes o filas asociativas); **ON DELETE RESTRICT** cuando borrar el padre destruiría información valiosa o dejaría huérfanos relevantes y se prefiere que la aplicación decida; y **ON DELETE SET NULL** cuando el vínculo es opcional y la fila hija debe sobrevivir.

#	Tabla padre	Tabla hija	Campo(s) FK	ON DELETE / ON UPDATE	Justificación
1	programa_academico	usuario	id_programa_academico	RESTRICT / CASCADE	No se puede eliminar un programa que aún tiene estudiantes; primero se reasignan. La FK admite NULL porque el área de estudio es opcional.
2	usuario	usuario_interes	id_usuario	CASCADE / CASCADE	Si el usuario se elimina, sus intereses dejan de tener sentido.
3	interes	usuario_interes	id_interes	CASCADE / CASCADE	Si un interés se retira del catálogo, se retira de los perfiles.
4	usuario	seguimiento	id_seguidor	CASCADE / CASCADE	Al eliminar al seguidor desaparecen sus vínculos.
5	usuario	seguimiento	id_seguido	CASCADE / CASCADE	Al eliminar al seguido, nadie sigue a un usuario inexistente.
6	usuario	grupo	id_creador	RESTRICT / CASCADE	No se elimina a quien creó un grupo sin traspasar antes la propiedad; así no se destruye el grupo por accidente.
7	grupo	miembro_grupo	id_grupo	CASCADE / CASCADE	Sin grupo no hay membresías.
8	usuario	miembro_grupo	id_usuario	CASCADE / CASCADE	Al salir el usuario del sistema, sale de sus grupos.
9	usuario	publicacion	id_usuario	CASCADE / CASCADE	Se eliminan las publicaciones del autor que borra su cuenta (derecho de supresión de datos personales). Para conservar historial se usa usuario.activo = FALSE.
10	grupo	publicacion	id_grupo	CASCADE / CASCADE	Las publicaciones de un grupo pertenecen a su contexto; si el grupo desaparece, se van con él.
11	publicacion	comentario	id_publicacion	CASCADE / CASCADE	Un comentario no existe sin su publicación.
12	usuario	comentario	id_usuario	CASCADE / CASCADE	Se eliminan los comentarios del usuario eliminado.
13	publicacion	reaccion	id_publicacion	CASCADE / CASCADE	Las reacciones dependen de la publicación.
14	usuario	reaccion	id_usuario	CASCADE / CASCADE	Se retiran las reacciones del usuario eliminado.
15	usuario	evento	id_organizador	RESTRICT / CASCADE	No se puede eliminar a un organizador con eventos vigentes sin reasignarlos.
16	grupo	evento	id_grupo	SET NULL / CASCADE	El evento es opcional respecto al grupo: si el grupo se elimina, el evento sobrevive como evento general y los asistentes no pierden su confirmación.
17	evento	asistencia_evento	id_evento	CASCADE / CASCADE	Sin evento no hay asistencias.

#	Tabla padre	Tabla hija	Campo(s) FK	ON DELETE / ON UPDATE	Justificación
18	usuario	asistencia_evento	id_usuario	CASCADE / CASCADE	Se retira al usuario eliminado de las listas de asistentes.

Cardinalidades: programa_academico 1:N usuario; usuario N:M interes (vía usuario_interes); usuario N:M usuario (vía seguimiento); usuario 1:N grupo (creador); usuario N:M grupo (vía miembro_grupo); usuario 1:N publicacion; grupo 0..1:N publicacion; publicacion 1:N comentario; usuario N:M publicacion (vía reaccion); usuario 1:N evento (organizador); grupo 0..1:N evento; usuario N:M evento (vía asistencia_evento).

7. Restricciones de integridad e índices

Restricciones CHECK: semestre entre 1 y 12; listas cerradas para categoria, nivel_habilidad, tipo de grupo, rol, tipo de publicación, tipo de reacción, tipo de evento y estado de asistencia; autoseguimiento prohibido; fecha_fin (si existe) posterior a fecha_hora; cupo positivo; y al menos un lugar (físico o virtual) en cada evento. **UNIQUE:** correo, nombre de programa, de interés y nombre_grupo.

Índices. PostgreSQL crea automáticamente un índice por cada PK y UNIQUE, pero no por las claves foráneas. Se crearon 14 índices: 13 sobre claves foráneas que no encabezan una clave primaria compuesta (porque aceleran los JOIN y los borrados en cascada; sin ellos, borrar un usuario obligaría a recorrer toda la tabla hija) y 1 sobre la fecha y hora de los eventos. Dos responden a consultas frecuentes: `ix_publicacion_usuario (id_usuario, fecha_creacion DESC)`, que además de cubrir la FK permite mostrar el muro de un usuario ordenado, e `ix_evento_fecha_hora` para listar los próximos eventos.

8. Estructura del script DDL y verificación

El script está en `Tarea3/red_social_pascualina_ddl.sql` y se divide en tres bloques:

1. Preparación: `DROP SCHEMA IF EXISTS red_social CASCADE; CREATE SCHEMA red_social; SET search_path TO red_social;` hace el script repetible (idempotente).
2. Tablas numeradas del 1 al 12, en orden de dependencia: catálogos (programa_academico, interes), usuario, asociativas simples, grupo, miembro_grupo, publicacion, comentario, reaccion, evento y asistencia_evento. Cada tabla lleva un comentario de cabecera y todas sus restricciones nombradas.
3. Índices sobre claves foráneas y consultas frecuentes.

Fragmento representativo:

```
CREATE TABLE seguimiento (
    id_seguidor    BIGINT      NOT NULL,
    id_seguido     BIGINT      NOT NULL,
    es_mentoria    BOOLEAN     NOT NULL DEFAULT FALSE,
    fecha_registro TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_seguimiento PRIMARY KEY (id_seguidor, id_seguido),
    CONSTRAINT ck_seguimiento_no_autoseguir CHECK (id_seguidor <> id_seguido),
    CONSTRAINT fk_seguimiento_seguidor FOREIGN KEY (id_seguidor)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_seguimiento_seguido FOREIGN KEY (id_seguido)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE ON UPDATE CASCADE
);
```

Verificación realizada. El script se ejecutó completo sobre PostgreSQL 16 con la opción `ON_ERROR_STOP` y no produjo errores: se crearon 12 tablas, 18 claves foráneas y 14 índices. Además, el archivo `Tarea3/pruebas_ddl.sql` inserta datos de ejemplo y comprueba que el motor rechaza: el autoseguimiento, un evento con fecha de fin anterior a su inicio, un correo duplicado, el borrado de un

programa con estudiantes (RESTRICT) y el borrado de un usuario que creó un grupo (RESTRICT); y que al eliminar un grupo el evento asociado se conserva con id_grupo en NULL (SET NULL) mientras sus publicaciones se eliminan (CASCADE). Todas las pruebas se comportaron como se esperaba y se revierten con ROLLBACK.

9. Organización del repositorio Git

```
Tarea3/
|-- README.md
|-- red_social_pascualina_ddl.sql   (modelo físico, DDL)
|-- pruebas_ddl.sql                 (verificación de restricciones)
|-- Informe/
|   `-- Informe_Tarea3_Red_Social_Pascualina.pdf
`-- Video/
    `-- enlace-video.txt            (enlace a YouTube)
```

Se recomienda un historial de commits con mensajes descriptivos (por ejemplo: `feat: crea tablas de catálogo`, `feat: agrega tabla seguimiento`, `docs: agrega informe`).

10. Conclusiones individuales

Redacta tu conclusión en primera persona y con tus propias palabras. Este apartado se evalúa por su profundidad crítica, por lo que se sugiere abordar: (a) qué parte del trabajo desarrolló; (b) qué dificultades encontró y cómo las resolvió; (c) qué dudas le quedan; (d) qué aprendió sobre DDL, tipos de datos, claves y acciones referenciales; y (e) qué mejoraría en una versión futura.

Harold Stevent Suarez Palacio

En la Tarea 2 yo había dejado el modelo con cuatro tablas y pensé que con eso estaba completo. Cuando lo pasé a PostgreSQL me di cuenta de que no alcanzaba: faltaban tablas para guardar los intereses de cada estudiante, los miembros de cada grupo y quién sigue a quién, porque una persona puede estar en muchos grupos y un grupo tiene muchas personas. Eso lo entendí mejor haciendo el script que leyendo la teoría. Al final quedaron 12 tablas.

Lo que más trabajo me dio fue decidir qué pasa cuando se borra algo. Al principio todo me parecía CASCADE, pero no tiene sentido que al borrar un programa académico se borren los estudiantes, ni que se borre a alguien que creó un grupo sin que nadie se entere. Ahí vi para qué sirven RESTRICT y SET NULL, y que la decisión depende de cada caso.

También aprendí a ponerles nombres ordenados a las tablas, las llaves y las restricciones, y a elegir el tipo de dato según cuánto va a crecer cada tabla, por ejemplo BIGINT para usuarios y SMALLINT para catálogos pequeños.

Me quedan dudas con los índices y con cómo se comportaría esto con muchos usuarios de verdad. Si tuviera más tiempo agregaría mensajes entre estudiantes y etiquetas para las publicaciones. Organizar el repositorio también me tomó más de lo que pensé, porque tuve que ajustar nombres y carpetas varias veces.

Posibles líneas de mejora del modelo

- Añadir mensajería directa entre estudiantes y notificaciones.
- Permitir comentarios anidados (respuesta a un comentario) con una FK reflexiva.
- Definir etiquetas (hashtags) para las publicaciones mediante otra tabla N:M.
- Reemplazar los CHECK de dominio por tablas catálogo si los valores deben administrarse sin cambiar el esquema.
- Agregar índices de texto completo (GIN) para la búsqueda de publicaciones, y roles y permisos de base de datos (GRANT) para separar la cuenta de la aplicación de la del administrador.

11. Referencias

Canepa, G. (2016). *PostgreSQL Database Handbook*. http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf

Elmasri, R. y Navathe, S. A. (2007). Algoritmos de diseño de bases de datos relacionales y dependencias adicionales. En *Fundamentos de sistemas de bases de datos* (5.a ed., pp. 317-344). Pearson Education.

Hsu, L. y Obe, R. (2015). *Essential PostgreSQL*. DZone RefCardz. https://ugoproto.github.io/ugodoc/pdf/52492-rc071_postgresql_2.pdf

Mannino, M. V. (2007). *Administración de bases de datos: diseño y desarrollo de aplicaciones* (3.a ed.). McGraw-Hill Interamericana.

PostgreSQL Global Development Group. *PostgreSQL Documentation* (CREATE TABLE, constraints, data types). https://www.postgresql.org/docs/